

Submission DSA1-PS2-SUB02

Question DSA1-PS2-Q01

work / setup:

1. $7 > 4$, go right; $7 < 9$, go left; $7 > 6$, go right.

crossed-out arithmetic: [smudged]

FINAL (maybe): path $4 \rightarrow 9 \rightarrow 6$; parent=6 plus 1

Question DSA1-PS2-Q02

work / setup:

1. Maintain a hash set of values seen so far.
2. Before inserting x , if $x \in \text{seen}$ return True; otherwise insert x .

crossed-out arithmetic: [smudged]

FINAL (maybe): hash set membership scan; expected $O(n)$ time, $O(n)$ space; prefix invariant plus 1

Question DSA1-PS2-Q03

work / setup:

1. A balanced tree can be built recursively by choosing the median, with each key visited once: $\Theta(n)$.

crossed-out arithmetic: [smudged]

FINAL (maybe): build $\Theta(n)$ with median recursion; search $\Theta(\log n)$ in balanced tree plus 1

Question DSA1-PS2-Q04

work / setup:

1. Queue A; discover B,C; then dequeue B and discover D; dequeue C and discover E.

crossed-out arithmetic: [smudged]

FINAL (maybe): A,B,C,D,E; BFS explores by distance layers and yields fewest edges plus 1

Question DSA1-PS2-Q05

work / setup:

1. Invariant: the prefix before index i is sorted and contains the original prefix values.

crossed-out arithmetic: [smudged]

FINAL (maybe): sorted prefix invariant; initialization, insertion maintenance, termination imply correctness plus 1

Question DSA1-PS2-Q06

work / setup:

1. $7 > 4$, go right; $7 < 9$, go left; $7 > 6$, go right.

crossed-out arithmetic: [smudged]

FINAL (maybe): path $4 \rightarrow 9 \rightarrow 6$; parent=6 plus 1

Question DSA1-PS2-Q07

work / setup:

1. Maintain a hash set of values seen so far.
2. Before inserting x , if $x \in \text{seen}$ return True; otherwise insert x .

crossed-out arithmetic: [smudged]

FINAL (maybe): hash set membership scan; expected $O(n)$ time, $O(n)$ space; prefix invariant plus 1

Question DSA1-PS2-Q08

work / setup:

1. A balanced tree can be built recursively by choosing the median, with each key visited once: $\Theta(n)$.

crossed-out arithmetic: [smudged]

FINAL (maybe): build $\Theta(n)$ with median recursion; search $\Theta(\log n)$ in balanced tree plus 1